

RAG Workshop Guide

How to Build a RAG Agent in n8n | AI Snack Club | May 29, 2026

1. What is RAG?

RAG stands for Retrieval-Augmented Generation. It is a technique used to improve AI language models by allowing them to retrieve external information before generating a response. Instead of relying only on what the model learned during training, a RAG system first searches a knowledge source — such as documents, databases, websites, PDFs, or internal company files — and then uses the retrieved information to answer the user's question. This makes responses more accurate, current, and grounded in real data.

The Open-Book Exam Analogy

One way to think about RAG is as an open-book exam. A normal language model answers from memory alone — like a student who can only use what they memorized the night before. A RAG system, on the other hand, is allowed to look up relevant material in a textbook or library before answering. That retrieval step is what makes RAG systems far more useful and reliable in practical applications.

How It Works Step by Step

1. **Collect documents** into a knowledge base — PDFs, Word docs, web pages, internal files.
2. **Chunk the documents** into smaller sections because large documents are difficult for language models to process all at once.
3. **Create embeddings** by converting each chunk into a numerical representation that captures its semantic meaning.
4. **Store in a vector database** designed to quickly find semantically similar content.
5. **At query time**, the user's question is also converted into an embedding and compared against stored document embeddings.
6. **Retrieve and generate**: the most relevant chunks are inserted into the prompt, and the model generates an answer grounded in that context.

2. RAG vs. Fine-Tuning vs. Prompt Engineering

These three techniques are often confused. Here is when to use each one.

Prompt Engineering

Best for: Simple tasks, one-off queries, getting started quickly.

Limitation: You are limited to what the model already knows. No access to your private or updated data.

Fine-Tuning

Best for: Teaching the model a new style, tone, or domain-specific behavior.

Limitation: Expensive, slow to update, does not reliably inject new factual knowledge. A fine-tuned model can still hallucinate facts it was trained on.

RAG

Best for: Answering questions from your own documents, keeping knowledge current without retraining, and reducing hallucinations.

Limitation: Retrieval quality depends on how well documents are chunked and embedded. Poorly structured documents produce poor answers.

Rule of thumb: if you need the model to know something specific that is not in its training data, use RAG. If you need it to behave differently, consider fine-tuning. If you just need it to follow instructions better, start with prompt engineering.

3. Common RAG Use Cases

RAG is widely used across industries wherever people need AI to answer questions from specific, controlled knowledge sources.

Customer Support

A company uploads its product documentation, return policies, and FAQs. Customers ask questions and get instant, accurate answers grounded in actual policy documents — not hallucinated ones.

Internal Knowledge Base

Employees ask questions about HR policies, onboarding processes, or internal procedures. Instead of digging through shared drives, they get direct answers with source citations pointing to the exact document.

Legal and Compliance Research

Lawyers or compliance teams upload contracts, regulations, and case files. The RAG system surfaces relevant clauses or precedents in response to natural language queries.

Personal Productivity

Upload your meeting notes, saved articles, and research documents. Ask questions like "what did we decide about the Q3 roadmap?" and get answers pulled from your own notes.

4. How to Evaluate If Your RAG Is Working

Building a RAG system is only half the work. Knowing whether it is actually retrieving the right information is the other half.

Check the Source Citations

A well-built RAG system always tells you which document it pulled from. If the answer does not match the cited source, your retrieval is broken — not your LLM.

Test With Questions You Know the Answers To

Before deploying, create a small set of test questions with known correct answers from your documents. Run them through the system. If the answers are wrong, check whether the right chunks are being retrieved before blaming the model.

Watch the Confidence Threshold

Most vector databases return a similarity score with each result. If your system answers confidently on a low-score retrieval, it is likely hallucinating. Set a minimum confidence threshold and return "I don't have that information" when nothing meets it.

5. Common Mistakes When Building RAG Systems

Most RAG failures are not model failures. They are data and architecture failures.

Poor Document Quality

Documents full of tables and images with no text descriptions, or files with inconsistent formatting all produce bad embeddings. Garbage in, garbage out. Clean your source documents before ingesting them.

Chunks That Are Too Large or Too Small

Chunks that are too large dilute the relevant content. Chunks that are too small lose context. A common starting point is 300 to 500 tokens per chunk with some overlap between chunks so context is not lost at boundaries.

Not Filtering Low-Confidence Results

If you pass every retrieved chunk to the LLM regardless of similarity score, the model will try to answer using irrelevant content. Always filter by a minimum confidence threshold before passing context to the model.

Treating RAG as a Set-and-Forget System

Your knowledge base goes stale. Documents get updated, policies change, new information becomes relevant. Build a process for keeping your vector store updated — either through scheduled re-ingestion or triggered updates when documents change.

Skipping Evaluation

Most people build a RAG system, ask it two questions that work, and call it done. Real evaluation means testing edge cases, questions outside your knowledge base, and questions that require synthesizing information from multiple documents.

This document was created for the AI Snack Club workshop on May 29, 2026.

*Presenter: **Jessica Singh** | github.com/jessica-singh79/rag-knowledge-assistant*